

Terraform en AWS: Workshop de Fundamentos

¿Qué vamos a construir?

En este workshop vamos a aprender los fundamentos de Terraform mientras construimos infraestructura real en AWS paso a paso. Al final del laboratorio práctico, cada uno tendrá un juego corriendo en la nube, accesible desde el navegador en `<su-nombre>.com`.

El juego es libre — puede ser Super Mario, Doom, Tetris, o cualquier otro que encuentren como imagen Docker pública en [Docker Hub](#). La única condición es que corra en un contenedor y sea accesible desde el navegador. Cada quien usará su dominio personal (`<su-nombre>.com`) que ya tienen registrado.

El objetivo no es solo que el juego funcione — es que entiendan **por qué** cada recurso existe, cómo se conectan entre sí, y cómo Terraform nos permite construir infraestructura de forma repetible, segura y colaborativa.

Lo que vamos a crear:

- Una **VPC** con subnets pública y privada
- Un **Internet Gateway** y tabla de rutas
- Un **Security Group** como firewall de la instancia
- Un repositorio **ECR** para almacenar la imagen Docker del juego
- Un **IAM Role** para dar permisos seguros a la instancia
- Una instancia **EC2** que descarga y corre el juego automáticamente
- Un **registro DNS en Route53** para apuntar su dominio personal (`<su-nombre>.com`) a la IP pública de su instancia EC2 y acceder al juego desde el navegador

Lo que van a aprender:

- Cómo funciona el estado de Terraform y por qué es importante
- La diferencia entre estado local y estado remoto en S3
- Qué son los data sources y cómo leer infraestructura existente
- Cómo conectar recursos entre sí usando referencias de Terraform
- La diferencia entre subnets públicas y privadas
- Por qué usamos IAM Roles en lugar de credenciales en el servidor
- Cómo conectarse a una instancia EC2 de forma segura sin SSH usando Session Manager

Parte 1: Configuración Inicial

1. Configurar Credenciales de AWS

Antes de usar Terraform necesitamos autenticarnos con AWS creando un usuario IAM con acceso programático.

- Ingresar a la consola de AWS → IAM → Crear nuevo usuario
- Adjuntar la política `AdministratorAccess`
- Generar un Access Key ID y Secret Access Key — nunca compartas tu Secret Key ni la subas a Git
- Instalar la extensión AWS Toolkit en VS Code
- Configurar las credenciales ejecutando:

```
aws configure
```

Ingresar el Access Key ID, Secret Access Key, región por defecto (`us-east-2`) y formato de salida (`json`).

Para guardar un perfil con nombre (recomendado cuando se trabaja con múltiples cuentas de AWS):

```
aws configure --profile <nombre-del-perfil>
```

Verificar que el perfil se guardó correctamente:

```
aws sts get-caller-identity --profile <nombre-del-perfil>
```

Deberías ver tu `UserId` , `Account` y `Arn` :

```
{
  "UserId": "AIDAZTKRQBD6IW4I4ZGJW",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/tu-usuario"
}
```

Agregar el perfil al bloque provider de Terraform:

```
provider "aws" {
  region  = "us-east-2"
  profile = "<nombre-del-perfil>"
}
```

Nunca subas credenciales ni archivos `.tfvars` a Git. Siempre agrégalos al `.gitignore` .

2. Instalar y Verificar Terraform

Prerequisitos:

- Una terminal
- Una cuenta activa de AWS
- VS Code con la extensión HashiCorp Terraform instalada

Instalación:

```
# macOS
brew install terraform

# Windows (Chocolatey)
choco install terraform

# Linux
sudo apt install terraform
```

Verificar la instalación:

```
terraform version
```

Administrar múltiples versiones de Terraform con tfenv:

```
tfenv list      # ver todas las versiones instaladas
tfenv install 1.6.0 # instalar una versión específica
tfenv use 1.6.0  # cambiar a una versión
```

Parte 2: Comandos Principales

Terraform usa **HashiCorp Configuration Language (HCL)** — un lenguaje declarativo donde describes *qué* quieres y Terraform determina *cómo* lograrlo.

Comando	Propósito
terraform init	Inicializa el directorio de trabajo y descarga los plugins del provider
terraform fmt	Formatea los archivos .tf al estilo canónico
terraform validate	Verifica la sintaxis sin conectarse a AWS
terraform plan	Muestra exactamente qué va a cambiar sin tocar la infraestructura real
terraform apply	Crea, actualiza o destruye recursos
terraform destroy	Elimina todos los recursos administrados por el estado actual

Siempre ejecutar `fmt` → `validate` → `plan` antes de `apply`.

Parte 3: Estado de Terraform

Terraform rastrea los recursos reales de AWS usando un archivo de estado (`terraform.tfstate`). Este archivo JSON es la fuente de verdad de Terraform — mapea tu código HCL a los recursos reales en la nube.

Por defecto el estado se guarda localmente. Para equipos, debe centralizarse en S3 para que todos trabajen desde la misma fuente de verdad.

```
terraform {
  required_version = ">= 1.10.0"

  backend "s3" {
    bucket      = "nombre-del-bucket"
    key         = "mario/terraform.tfstate"
    region      = "us-east-2"
    encrypt     = true
    use_lockfile = true
  }
}
```

A partir de Terraform 1.10, `use_lockfile = true` habilita el bloqueo nativo de estado en S3 — no se requiere tabla DynamoDB. Siempre usar `encrypt = true`.

Parte 4: Providers

Un provider es un plugin que permite a Terraform comunicarse con una plataforma específica (AWS, Azure, GCP, etc.). Al ejecutar `terraform init`, Terraform descarga el provider definido en tu configuración.

```
terraform {
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = "us-east-2"
}
```

Parte 5: Variables, Locals y Outputs

Para mantener el código de Terraform reutilizable y evitar valores hardcoded, usamos variables, locals y outputs.

Concepto	Descripción
Variables	Entradas que se pasan a la configuración desde afuera (como parámetros de una función)
Locals	Valores internos contruidos a partir de expresiones — no pueden ser sobreescritos desde afuera
Outputs	Valores exportados después del apply, usados para mostrar información o pasar datos entre estados

Variable:

```
variable "instance_type" {
  description = "Tipo de instancia EC2"
  type        = string
  default     = "t3.micro"
}
```

Local:

```
locals {
  resource_name = "${var.env}-${var.project}"
}
```

Outputs:

```
output "instance_public_ip" {
  description = "IP pública de la instancia EC2"
  value       = aws_instance.my_server.public_ip
}

output "vpc_id" {
  description = "ID de la VPC principal"
  value       = aws_vpc.main.id
}

output "bucket_name" {
  description = "Nombre del bucket S3"
  value       = aws_s3_bucket.assets.bucket
}

output "db_password" {
  description = "Contraseña maestra de RDS"
  value       = aws_db_instance.main.password
  sensitive   = true
}
```

Después de `terraform apply`, recuperar outputs con `terraform output` o `terraform output <nombre>`.

Archivo `.tfvars` — guardar los valores de las variables aquí para no escribirlos en cada apply:

```
# terraform.tfvars
environment = "dev"
instance_type = "t3.micro"
```

Agregar `terraform.tfvars` al `.gitignore` si contiene valores sensibles.

Parte 6: Imports y Configuration Drift

Importar Recursos No Administrados

Un recurso creado manualmente en la consola de AWS no está administrado por Terraform. Para traerlo bajo control de Terraform:

Opción A — Bloque import (HCL):

```
import {
  to = aws_instance.my_server
  id = "i-1234567890abcdef0"
}
```

Opción B — CLI:

```
terraform import aws_instance.my_server i-1234567890abcdef0
```

Después de importar, escribir el HCL correspondiente en los archivos `.tf`, luego ejecutar `terraform plan` — no debería mostrar cambios si el código coincide con la realidad.

Configuration Drift

Si alguien modifica manualmente un recurso administrado por Terraform en la consola de AWS, la infraestructura real ya no coincide con el código. Esto se llama **configuration drift**.

Detectar drift con:

```
terraform plan -refresh-only
```

Terraform mostrará qué cambió e intentará revertirlo en el próximo `apply` para que coincida con el HCL.

Tu repositorio es la fuente de verdad. Siempre lleva los cambios manuales de vuelta al código.

Parte 7: .gitignore para Proyectos Terraform

```
.terraform/  
terraform.tfstate  
terraform.tfstate.backup  
*.tfvars  
.terraform.lock.hcl
```

Parte 8: Laboratorio Práctico — Deploy de tu Juego en AWS

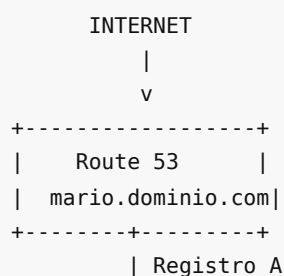
Al final de este laboratorio cada uno tendrá un juego corriendo y accesible en `<tudominio>.com`.

El juego es libre — busca en [Docker Hub](#) una imagen de cualquier juego que corra en el navegador. Anota el nombre de la imagen y el puerto en el que corre, los necesitarás en el Paso 6.

Cada paso indica qué recurso necesitan crear y qué debe hacer. Busquen el recurso correspondiente en el [Registro de Terraform](#) y escriban el código HCL necesario.

Tip para encontrar tu juego: Busca en Docker Hub con términos como `browser game`, `html5 game`, o el nombre del juego directamente. Verifica que la imagen tenga instrucciones claras de en qué puerto corre.

Arquitectura



- ```

 v
 +-----+
 | Internet Gateway |
 +-----+
 |
+-----v-----+
| VPC 10.0.0.0/16 |
|
| +-----+
| | Subnet Pública 10.0.1.0/24 |
| |
| | +-----+
| | | Security Group
| | | puertos: 80, 8080
| | |
| | | +-----+
| | | | Instancia EC2
| | | | + IAM Role
| | | | (docker run)
| | | |
| | | +-----+
| | +-----+
| +-----+
|
| +-----+
| | Subnet Privada 10.0.2.0/24
| | (sin ruta a internet)
| +-----+
+-----+
|
| docker pull
 v
 +-----+
 | Repositorio ECR
 | (imagen Mario)
 +-----+

```

- Crear una **subnet privada** con CIDR `10.0.2.0/24` sin acceso a internet
- Crear un **Internet Gateway** y asociarlo a la VPC
- Crear una **tabla de rutas** que dirija todo el tráfico saliente ( `0.0.0.0/0` ) al Internet Gateway, y asociarla únicamente a la subnet pública
- Crear un **Security Group** que permita tráfico entrante en los puertos `80` y `8080` , y todo el tráfico saliente

*Recursos a investigar:* `aws_vpc` , `aws_subnet` , `aws_internet_gateway` , `aws_route_table` , `aws_route_table_association` , `aws_security_group`

---

### Paso 3 — Repositorio ECR

Necesitamos un lugar privado para almacenar la imagen Docker del juego.

- Crear un **repositorio ECR privado** llamado `mario-game`
- Habilitar el escaneo de imágenes al momento del push
- Agregar un output que muestre la URL del repositorio — la necesitarán en los siguientes pasos

*Recurso a investigar:* `aws_ecr_repository`

---

### Paso 4 — Subir la Imagen de Mario a ECR

Una vez creado el repositorio, ejecutar los siguientes comandos para subir la imagen del juego.

```
1. Obtener el ID de tu cuenta AWS
aws sts get-caller-identity --query Account --output text

2. Autenticar Docker con ECR
aws ecr get-login-password --region us-east-2 | \
 docker login --username AWS --password-stdin <account_id>.dkr.ecr.us-east-2.amazonaws.com

3. Descargar la imagen de Mario
docker pull pengbai/docker-supermario

4. Etiquetar la imagen para tu repositorio ECR
docker tag pengbai/docker-supermario <account_id>.dkr.ecr.us-east-2.amazonaws.com/mario-game:latest

5. Subir la imagen
docker push <account_id>.dkr.ecr.us-east-2.amazonaws.com/mario-game:latest
```

---

### Paso 5 — IAM Role para EC2

La instancia EC2 necesita permisos para descargar imágenes de ECR y para conectarse via Session Manager. La forma correcta de dar permisos a una instancia EC2 es a través de un IAM Role — nunca con credenciales hardcodedas.

- Crear un **IAM Role** con una política de confianza que permita al servicio `ec2.amazonaws.com` asumir el rol
- Adjuntar la política administrada `AmazonEC2ContainerRegistryReadOnly` para permitir descargar imágenes de ECR



- Adjuntar la política administrada `AmazonSSMManagedInstanceCore` para permitir conexiones via Session Manager
- Crear un **Instance Profile** que envuelva el rol — esto es lo que se adjunta directamente a la instancia EC2

*Recursos a investigar:* `aws_iam_role` , `aws_iam_role_policy_attachment` , `aws_iam_instance_profile`

---

## Paso 6 — Instancia EC2

Crear la instancia que va a correr el juego. La instancia debe:

- Usar el AMI obtenido del data source del Paso 1
- Ser de tipo `t3.micro`
- Estar en la **subnet pública**
- Tener el **Security Group** del Paso 2 asociado
- Tener el **Instance Profile** del Paso 5 asociado
- Usar `user_data` con el siguiente script de arranque que instala Docker, el agente de SSM, y corre el juego automáticamente:

```
#!/bin/bash
yum update -y
yum install -y docker amazon-ssm-agent
systemctl start docker
systemctl enable docker
systemctl start amazon-ssm-agent
systemctl enable amazon-ssm-agent
aws ecr get-login-password --region us-east-2 | \
 docker login --username AWS --password-stdin <ecr_repository_url>
docker pull <ecr_repository_url>:latest
docker run -d -p 8080:8080 <ecr_repository_url>:latest
```

- Agregar un output con la IP pública de la instancia

*Recurso a investigar:* `aws_instance`

---

## Paso 7 — Registro DNS en Route53

El último paso es apuntar el dominio a la instancia EC2.

- Crear un **registro tipo A** en la zona hospedada de Route53 que ya tienen registrada con su nombre personal
- El nombre del registro debe ser `juego.<su-nombre>.com` — reemplazar `juego` por el nombre del juego que eligieron
- El registro debe apuntar a la **IP pública de su instancia EC2** del Paso 6
- Usar un TTL de `300` segundos
- Agregar un output con la URL completa del juego

*Recurso a investigar:* `aws_route53_record`

---

## Verificar

```
Obtener la URL del juego
terraform output mario_url
```

Para conectarse a la instancia EC2 usamos **AWS Session Manager** — sin puerto 22, sin llaves SSH.

Conectarse desde la consola de AWS:

- EC2 → Instancias → seleccionar la instancia → Conectar → Session Manager → Conectar

O desde la CLI:

```
aws ssm start-session --target <instance-id> --profile lab
```

Una vez conectado, verificar que Docker está corriendo:

```
docker ps
```

Abrir la URL en el navegador — tu juego debería cargar. 🎮